

```

        if triangles[i][k] == triangles[j][n]:
            commonpoints += 1
    if commonpoints == 2:
        lines.append([list(centers[i][0]),list(centers[j][0])])

```

This code will be added to our plotting function, since our final goal is a plotted Voronoi diagram.

While we're at it, we can make several other useful additions to our plotting function. The new plotting function is shown in [Listing 7-4](#), with the changes in bold:

```

def plot_triangle_circum(triangles,centers,plotcircles,plotpoints,
\plottriangles,plotvoronoi,plotvpoints,thename):
    fig, ax = pl.subplots()
    ax.set_xlim([-0.1,1.1])
    ax.set_ylim([-0.1,1.1])

    lines=[]
    for i in range(0,len(triangles)):
        triangle = triangles[i]
        center = centers[i][0]
        radius = centers[i][1]
        itin = [0,1,2,0]
        thelines = genlines(triangle,itin)
        xs = [triangle[0][0],triangle[1][0],triangle[2][0]]
        ys = [triangle[0][1],triangle[1][1],triangle[2][1]]

        lc = mc.LineCollection(genlines(triangle,itin), linewidths=2)
        if(plottriangles):
            ax.add_collection(lc)
        if(plotpoints):
            pl.scatter(xs, ys)

        ax.margins(0.1)

    ❶ if(plotvpoints):
        pl.scatter(center[0],center[1])

        circle = pl.Circle(center, radius, color = 'b', fill = False)
        if(plotcircles):
            ax.add_artist(circle)

    ❷ if(plotvoronoi):

```